RecipeFinder — Version 1 Review

Student: Yanxue Liu
Course: HCI 5840
Project: RecipeFinder
Date: October 20, 2026

1. Project Overview

RecipeFinder is a Python application that helps users decide what to cook based on ingredients they already have at home.

Users can enter a list of ingredients, and the program searches a recipe database to find matching recipes. It calculates a match percentage for each recipe and shows which ingredients are missing.

The goal of this project is to help users find meal ideas more easily and reduce food waste.

For Version 1, I focused on developing the core functions using a command-line interface (CLI).

2. Current Implementation

So far, I have implemented the following features:

1. Ingredient Input

Users can enter ingredients separated by commas. The program processes the input by removing extra spaces and converting the text to lowercase.

2. Ingredient Matching

The program compares the user's ingredients with recipes stored in a CSV file. It also uses spelling correction to help users when they enter an ingredient incorrectly.

3. Match Percentage Calculation

The program calculates how many required ingredients the user already has.

For example, if a recipe requires four ingredients and the user has three of them, the match percentage is 75%.

4. Recipe Recommendations

The program displays all recipes and sorts them by match percentage, from highest to lowest.

Each recommendation includes the recipe name, match percentage, and missing ingredients.

5. Recipe Selection

Users can select a recipe by entering its number. The program then displays the selected recipe's name, match percentage, and missing ingredients.

Current Application Demonstration

I tested the program by entering the following ingredients:

egg, tomato, onion, salt, oil

The program displayed the following recommendations:

Recipe	**Match Percentage**
Tomato Egg Stir Fry	100%
Potato Omelette	75%
Chicken Vegetable Soup	40%
Egg Fried Rice	25%
Tomato Pasta	25%

The program also displayed the missing ingredients for each recipe.

When I selected Recipe 1, the program displayed:

```
liuyanxue@liuyanxuedeMBP RecipeFinder % python3 main.py
Welcome to RecipeFinder!
Enter the ingredients you have: egg, tomato, onion, salt, oil
You entered: egg, tomato, onion, salt, oil
Ingredient list: ['egg', 'tomato', 'onion', 'salt', 'oil']
Matched ingredients: ['egg', 'tomato', 'onion', 'salt', 'oil']

===== Recipe Recommendations =====

1. Tomato Egg Stir Fry
   Match: 100%
   Missing ingredients: None

2. Potato Omelette
   Match: 75%
   Missing ingrediends: potato

3. Chicken Vegetable Soup
   Match: 40%
   Missing ingrediends: chicken, carrot, potato

4. Egg Fried Rice
   Match: 25%
   Missing ingrediends: rice, green onion, soy sauce

5. Tomato Pasta
   Match: 25%
   Missing ingrediends: pasta, garlic, olive oil

6. Chicken Salad
   Match: 25%
   Missing ingrediends: chicken, lettuce, cucumber

7. Cheese Omelette
   Match: 25%
   Missing ingrediends: cheese, milk, butter

8. Chicken Fried Rice
   Match: 20%
   Missing ingrediends: chicken, rice, carrot, soy sauce

9. Garlic Butter Pasta
   Match: 0%
   Missing ingrediends: pasta, garlic, butter, cheese

10. Vegetable Stir Fry
   Match: 0%
   Missing ingrediends: broccoli, carrot, garlic, soy sauce
```

Figure 1. Recipe recommendations based on user-entered ingredients.

```
Enter a recipe number to view details: 1

===== Selected Recipe =====
Recipe: Tomato Egg Stir Fry
Match: 100%
Missing ingredients: []
liuyanxue@liuyanxuedeMBP RecipeFinder %
```

Figure 2. Selected recipe details displayed in the terminal.

These results show that the basic recipe matching and recommendation functions are working.

3. Issues Encountered and Solutions

During development, I encountered several challenges.

Issue 1: Python Indentation

One challenge was understanding Python indentation, especially when using `for` loops and `if` statements.

At first, some code was placed at the wrong indentation level, which caused incorrect results.

I fixed this by checking the structure of my loops and making sure each calculation was inside the correct block.

Issue 2: Sorting Recipe Recommendations

Initially, the program displayed recipes in their original order in the CSV file.

However, I wanted users to see the recipes with the highest match percentages first.

I solved this by storing the recipe results in a list and using Python's `sort()` function with `reverse=True`.

Issue 3: Displaying Results Clearly

At first, the program printed the recipe results as a Python list containing dictionaries. This was difficult to read.

I improved the output by using a `for` loop to display each recipe separately, including its name, match percentage, and missing ingredients.

This made the recommendations easier for users to understand.

4. Unresolved Issues

Although the core functions are working, several features still need improvement.

1. Input Validation

Currently, the program does not fully handle invalid inputs. For example, if a user enters a letter instead of a recipe number, the program may produce an error.

I plan to add input validation to prevent these problems.

2. Recipe Details

The program currently displays basic information about the selected recipe, but it does not show the complete cooking instructions.

I plan to add the full ingredient list and cooking steps.

3. Repeated Searches

Currently, users need to restart the program to search with different ingredients.

I would like to add an option that allows users to start another search without restarting the application.

5. Help Needed

At this stage, I do not have any major problems preventing me from continuing the project.

However, I would appreciate feedback on how to organize my code as the application becomes more complex.

I am also interested in learning how to make the command-line interface easier to use.

6. Milestones for Version 2

For the next version, I plan to focus on completing the remaining features and improving the user experience.

Milestone	**Description**
1. Complete Recipe Details	Display the full ingredient list and cooking instructions when users select a recipe.
2. Improve User Interaction	Add input validation and allow users to search again without restarting the program.

3. Testing and Debugging	Test different ingredient combinations, spelling errors, and invalid inputs.
4. Improve the Interface (Optional)	Explore the possibility of creating a simple graphical interface if time permits.

My priority is to make sure the command-line version works reliably before adding more advanced features.

7. Self-Reflection

Am I satisfied with my progress so far?

Overall, I am satisfied with my progress because I have implemented the main functions of RecipeFinder.

At the beginning, I was not very familiar with Python programming. Building this application has helped me better understand how to use loops, conditions, lists, and dictionaries in a real project.

Can I finish the project in the next few weeks?

I think I can finish the core application in the remaining weeks.

The main matching and recommendation functions are already working. My next steps are to complete the recipe details, improve input validation, and test the application.

What was easier or more difficult than expected?

Reading recipe data from a CSV file was easier than I expected.

However, organizing the program logic was more challenging, especially when calculating match percentages, sorting recipes, and displaying results in the correct order.

I also realized that small changes in indentation can affect how the entire program works.

What did I learn?

One important thing I learned is that developing an application requires more than writing code that runs.

I also need to think about how users interact with the program and whether the results are easy to understand.

For example, sorting recipes by match percentage makes the recommendations more useful because users can quickly identify recipes they are most likely to make.

This project has helped me connect programming with my interest in user experience design.